

CampusVoice

College Grievance Redressal Portal — Complete Architecture, Team Division & Viva Guide

Role: Senior Full Stack Engineer & College Evaluator Guide | Team Size: 3-4 Members

1. High-Level Architecture Overview

CampusVoice is a full-stack web application designed to digitize and streamline college grievance resolution. It replaces physical complaint books with an auditable, transparent digital workflow supporting distinct personas: **Students** (who lodge and track complaints) and **Administrators** (who review and update complaint statuses).

End-to-End Architectural Data Flow

- **Step 1 (Client Submission):** Student fills out the grievance form on `submit.html` (Title, Category, Priority, Description).
- **Step 2 (Client Validation & Fetch):** JavaScript (`submit.js`) intercepts the submit event, validates non-empty inputs, and calls `data.js` (`createComplaint`).
- **Step 3 (HTTP Transmission):** An asynchronous POST request carrying a JSON payload and session cookies is transmitted to `/api/complaints`.
- **Step 4 (Controller & Auth Guards):** Flask checks `session['user_id']` via the `@login_required` decorator, re-validates payload bounds, and generates a formatted identifier (e.g., `CMP-100X`).
- **Step 5 (Database Persistence):** PyMySQL executes a parameterized INSERT query into MySQL, assigning `user_id` as a foreign key referencing the `users` table.
- **Step 6 (UI Feedback):** Server returns `{ success: true, complaint_id }`; the UI renders an immediate notification toast without a page reload.
- **Step 7 (Admin Lifecycle):** Admin logs in, views grievances via `GET /api/admin/complaints` (SQL INNER JOIN with `users`), and updates status via `PUT /api/admin/complaints/<id>/status`.

2. Member 1: Frontend Module

Role Summary: Responsible for User Interface, Client-Side State, DOM Manipulation, and API Fetching.

File & Component Breakdown

- `index.html` : Public landing page with hero section, quick categories, workflow cards, statistics, and FAQ accordion.
- `login.html` : Dual-tab authentication interface supporting both Sign In and Create Account forms.
- `dashboard.html` : Student portal with real-time metric cards (Total, Pending, In Progress, Resolved) and recent complaints list.
- `submit.html` : Structured grievance filing form with category and priority dropdowns.
- `complaints.html` : Complete student complaint history table with live search, status filtering, and priority filtering.
- `complaint-details.html` : Read-only timeline view displaying tracking code, student identity, timestamp, status badge, and official admin remarks.
- `admin.html` : Administrative dashboard with metrics, searchable complaint roster, and inline status update modal.
- `js/data.js` : Centralized API abstraction module containing `async/await` fetch helpers.
- `js/main.js` : Global layout controller managing active navigation links, session checks, and toast alerts.
- `css/style.css` : Mobile-responsive styling utilizing CSS variables, flexbox, CSS grid, and interactive hover states.

★ 2-MINUTE PRESENTATION SCRIPT FOR THE EXAMINER

"Good morning, Respected Evaluator. I was responsible for the Frontend and Client-Side Architecture of CampusVoice.

Our design goal was to build a clean, mobile-responsive, zero-dependency interface that provides students and administrators with intuitive navigation. I designed seven key pages using HTML5, modern CSS3, and JavaScript ES6. For students, we built dedicated views for registration, complaint submission, tracking history, and detailed status inspection. For staff, we built an administrative portal with filterable status tables.

Instead of scattering API calls across individual pages, I abstracted all backend communication into a centralized service module, `js/data.js`. Whenever an action occurs—such as lodging a complaint—the form validates input constraints client-side, builds a JSON payload, and initiates an asynchronous `fetch()` call. If the backend accepts the request, our UI immediately parses the JSON response and dynamically updates the DOM using templates without requiring a full page refresh. We also implemented an alert toast system that gives instant visual feedback for errors or success states.

Now I am ready to answer any questions regarding our frontend design, DOM manipulation, or client-side API integration."

Top 5 Frontend Viva Questions & Answers

Q1: Why did you use Vanilla JavaScript instead of a framework like React or Vue?

Direct Answer: Vanilla JavaScript was chosen to eliminate complex build tooling, `node_modules` dependencies, and bundle overhead. It directly leverages native browser DOM APIs and the `fetch` interface, providing instant load times, zero compilation requirements, and proving foundational software engineering principles without framework abstraction.

Q2: How do you prevent a user from submitting an empty or invalid complaint form?

Direct Answer: We implement two-tier validation. On the frontend in `submit.js`, we attach an event listener to form submission, call `event.preventDefault()`, and verify that title, category, priority, and description are non-empty. If invalid, an alert toast is shown immediately. If valid, data is sent to the server, which independently verifies the payload before database insertion.

Q3: How does your search and filter feature work on complaints.html?

Direct Answer: When complaints are fetched via `GET /api/complaints`, they are held in a client-side array. We listen to `input` and `change` events on the search box and dropdowns. On each trigger, an in-memory filter runs: checking if complaint titles match the query string and whether the status matches the selected filter. The table DOM is then immediately updated without repeated network round-trips.

Q4: How is sensitive data protected on the client side? Are passwords or tokens saved in localStorage?

Direct Answer: No passwords, tokens, or credentials are stored in `localStorage` or `sessionStorage`. Storing authentication tokens in `localStorage` leaves them vulnerable to Cross-Site Scripting (XSS) attacks. Instead, our application uses server-controlled HTTP session cookies handled transparently and securely by the browser.

Q5: What is the purpose of event.preventDefault() in your form handling?

Direct Answer: By default, submitting an HTML form initiates a traditional HTTP navigation that reloads the entire page. Calling `event.preventDefault()` suppresses this behavior, allowing our asynchronous JavaScript (`fetch`) to transmit data in the background and update the interface smoothly.

3. Member 2: Backend & API Module

Role Summary: Responsible for Server Setup, RESTful Routing, Controller Logic, Request Validation, and Error Handling.

API Routing & Controller Architecture

- **Server Framework:** Flask 3.x WSGI micro-framework written in Python.
- **Auth Endpoints:** `POST /api/auth/register`, `POST /api/auth/login`, `POST /api/auth/logout`, `GET /api/auth/me`
- **Student Endpoints:** `GET /api/complaints` (Student's own), `POST /api/complaints` (Lodge), `GET /api/complaints/<id>` (Details)
- **Admin Endpoints:** `GET /api/admin/complaints` (Master list with JOIN), `PUT /api/admin/complaints/<id>/status` (Update status & remarks)

★ 2-MINUTE PRESENTATION SCRIPT FOR THE EXAMINER

"Good morning, Respected Evaluator. I developed the Backend Server and RESTful API Layer of this system.

Our backend is built using Python and Flask. We structured the backend around REST design principles, establishing clear separation between user interfaces and data services. All business logic is centralized in `app.py`, which exposes structured endpoints under `/api/auth`, `/api/complaints`, and `/api/admin`.

Whenever a request reaches our server, it goes through our validation pipeline. First, our custom Python route decorators intercept the request to verify whether the client holds an active session and whether their role has permission to access that endpoint. Second, our controllers validate the incoming JSON payload to ensure required fields are present, properly formatted, and within acceptable limits.

For example, when an administrator changes a grievance status via `PUT /api/admin/complaints/<id>/status`, our backend validates that the requested status belongs to our permitted set—Pending, In Progress, Resolved, or Rejected—executes an update query via PyMySQL, and returns a structured JSON confirmation. Every database operation includes automated connection pooling, commit, and rollback logic to safeguard against errors.

I am pleased to demonstrate any route controller, middleware logic, or API response handling."

Top 5 Backend Viva Questions & Answers

Q1: What is a REST API and why did you choose it for this project?

Direct Answer: REST (Representational State Transfer) is an architectural style utilizing standard HTTP methods (`GET`, `POST`, `PUT`, `DELETE`) and exchanging stateless JSON payloads. We chose REST because it cleanly decouples the frontend user experience from server business logic, allowing either layer to be upgraded, tested, or scaled independently.

Q2: What are Python Decorators and how did you use them in your Flask code?

Direct Answer: A decorator is a design pattern that wraps an existing function to extend its behavior dynamically. We implemented `@login_required` to verify the existence of `session['user_id']` and `@admin_required` to verify `session.get('role') == 'admin'`. If unauthorized, requests are terminated immediately with HTTP 401 or 403 status codes.

Q3: Why do you use PUT instead of POST for updating a complaint status?

Direct Answer: Under HTTP/1.1 semantics, `POST` is intended for creating subordinate resources, whereas `PUT` is specified for updating or replacing an existing resource at a specific URI. Since updating a status modifies an existing record at `/api/admin/complaints/<id>/status`, `PUT` is semantically correct and idempotent.

Q4: How does Flask handle Cross-Origin Resource Sharing (CORS)?

Direct Answer: Because our Flask application serves both the static frontend files and the REST API endpoints from the same origin (host and port), CORS browser policies are not triggered. If the frontend were deployed on a different domain, we would configure the `flask-cors` extension to set `Access-Control-Allow-Origin` headers.

Q5: What happens if the database goes down while a user submits a complaint?

Direct Answer: The database connection attempt raises a `pymysql.MySQLError`. In our controller, this is intercepted inside a `try...except` block: any open transaction is rolled back via `conn.rollback()`, the error is logged, and the client receives a clean HTTP 500 JSON response (`{'error': 'Database unavailable'}`) without crashing the application.

4. Member 3: Database & Data Modeling

Role Summary: Responsible for Schema Normalization, Relational Modeling, Data Types, Constraints, and Persistence.

Relational Schema Definition

- **Table users :** `id` (PK, AUTO_INCREMENT), `name` (VARCHAR 100), `email` (VARCHAR 100 UNIQUE), `password` (VARCHAR 255), `role` (ENUM 'student', 'admin'), `created_at` (TIMESTAMP).
- **Table complaints :** `id` (PK), `complaint_id` (VARCHAR 20 UNIQUE), `user_id` (FK -> users.id), `title` (VARCHAR 200), `category` (ENUM), `priority` (ENUM), `description` (TEXT), `status` (ENUM), `admin_remarks` (TEXT), `created_at`, `updated_at`.

★ 2-MINUTE PRESENTATION SCRIPT FOR THE EXAMINER

"Good morning, Respected Evaluator. I was responsible for the Database Architecture, Relational Modeling, and Data Persistence.

For our storage engine, we chose MySQL because of its ACID transactional guarantees and robust foreign key integrity. Our database, `college_complaints`, contains two core relational entities: `users` and `complaints`.

When designing the schema, our primary objective was achieving Third Normal Form (3NF). We deliberately avoided storing redundant student information—such as student name or contact details—inside the complaints table. Instead, the complaints table holds a foreign key reference, `user_id`, linked to `users.id`. Whenever the admin portal needs student details, we execute an optimized `INNER JOIN` in MySQL. If a student ever updates their profile details, the change is reflected across all their past grievances automatically with zero data inconsistencies.

Furthermore, we enforced strict data integrity using SQL `ENUM` constraints for fields like grievance category, priority, and status. We also separated our database primary key—an auto-incrementing integer for index efficiency—from our human-readable tracking identifier, `complaint_id`. All connections use parameterized SQL queries through PyMySQL, completely safeguarding the database from SQL Injection.

I am ready to explain our schema constraints, normalization stages, or query executions."

Top 5 Database Viva Questions & Answers

Q1: What is Normalization and what Normal Form is your database in?

Direct Answer: Normalization organizes relational tables to eliminate redundancy and update anomalies. Our schema satisfies Third Normal Form (3NF): all columns contain atomic values (1NF), all attributes depend fully on the primary key without partial dependencies (2NF), and there are no transitive dependencies (3NF)—student details exist exclusively in the `users` table.

Q2: What happens if a user record is deleted? What is your Foreign Key strategy?

Direct Answer: In our schema, `FOREIGN KEY (user_id) REFERENCES users(id)` does not use `ON DELETE CASCADE`. Grievance records constitute formal institutional legal records and must never be deleted accidentally if an account is removed. The database prevents deletion of a user who has active complaints, preserving institutional history.

Q3: What is the difference between VARCHAR and TEXT in your schema?

Direct Answer: `VARCHAR(200)` stores variable-length strings inline up to 200 characters, ideal for short, indexable data like `title` and `complaint_id`. `TEXT` stores larger text blocks up to 64KB off-page with an internal pointer, making it ideal for complaint descriptions and admin remarks without bloating the primary row size.

Q4: Why did you use ENUM for Status and Category instead of plain VARCHAR?

Direct Answer: `ENUM` provides two critical benefits: first, it strictly restricts inputs to predefined valid options ('Pending', 'In Progress', 'Resolved', 'Rejected') at the database level. Second, MySQL internally stores ENUM values as 1-byte integer indices, improving query performance and storage efficiency.

Q5: How do you prevent SQL Injection at the database communication layer?

Direct Answer: We strictly utilize parameterized queries via PyMySQL using `%s` parameter placeholders. The SQL structure is compiled first, and parameters are transmitted as literal data rather than concatenated code, completely preventing SQL syntax injection.

5. Member 4 / Shared: Authentication & Role-Based Access

Role Summary: Responsible for Password Cryptography, Stateful Sessions, Route Authorization, and IDOR Prevention.

Security & Cryptographic Architecture

- **Password Hashing:** Passwords are never stored in plaintext or reversible encryption. We use Werkzeug's `generate_password_hash()` implementing the `bcrypt` algorithm with unique cryptographic salts.
- **Session Management:** Flask manages signed session cookies using a server-side `SECRET_KEY`. Cookies cannot be tampered with by the client.
- **IDOR Protection:** Enforced in controllers: Students can strictly query complaints `WHERE user_id = session['user_id']`. Students attempting to fetch another student's complaint ID receive HTTP 403 Forbidden.

★ 2-MINUTE PRESENTATION SCRIPT FOR THE EXAMINER

"Good morning, Respected Evaluator. I was responsible for Authentication, Cryptography, and Role-Based Access Control (RBAC).

Security is the backbone of any institution-grade portal. We focused on three critical security vectors: password protection, session security, and horizontal/vertical authorization.

First, we enforce zero plaintext password storage. When a student registers, their password is processed through Werkzeug's cryptographic engine using the `bcrypt` hashing algorithm with randomized salt generation. Even if an intruder obtained a raw dump of our database, the passwords cannot be reversed or decrypted via dictionary or rainbow-table attacks.

Second, we implemented stateful session management. Upon successful login, the server writes cryptographically signed session cookies. We created custom Python decorators—`@login_required` and `@admin_required`—that intercept requests to sensitive endpoints.

Third, we protected against Insecure Direct Object References (IDOR). A student cannot view another student's complaint simply by altering the URL or ID. Our backend always cross-checks the logged-in `session['user_id']` against the database record's owner before returning data. Admins, and only admins, have permission to trigger status updates via `/api/admin`.

I am prepared to explain our hashing algorithms, session lifecycle, or authorization code."

Top 5 Auth & Security Viva Questions & Answers

Q1: Why should passwords never be stored in plain text or encrypted with two-way encryption like AES?

Direct Answer: Plain text exposes credentials directly in case of a database breach. Two-way encryption relies on a private secret key; if that key is compromised, all passwords can be decrypted. In contrast, one-way cryptographic hashing (`bcrypt`) is irreversible: verification compares hashes rather than reconstructing passwords.

Q2: What is a Salt and why is it important in password hashing?

Direct Answer: A salt is a cryptographically random byte string generated and appended to the password before hashing. It ensures that identical passwords generate completely different hashes in the database, fully defeating precomputed rainbow-table and dictionary attacks.

Q3: What is the difference between Authentication and Authorization?

Direct Answer: Authentication (AuthN) verifies identity ("Who are you?"), such as checking credentials during login. Authorization (AuthZ) verifies permissions ("What are you permitted to do?"), such as confirming whether an authenticated user possesses the admin role to modify grievance statuses.

Q4: What is an IDOR vulnerability and how did you prevent it?

Direct Answer: IDOR (Insecure Direct Object Reference) occurs when an application exposes a record identifier (e.g., `/api/complaints/5`) without verifying ownership. We prevent this by verifying in `app.py`: `if session.get('role') != 'admin' and complaint['user_id'] != session['user_id']: return jsonify({'error': 'Access denied'}), 403`.

Q5: How does Flask protect against Session Tampering?

Direct Answer: Flask signs session cookies cryptographically using HMAC with the application's `SECRET_KEY`. If a client tampers with cookie contents (such as changing `role: 'student'` to `'admin'`), the cryptographic signature verification fails on the server, and Flask discards the session immediately.

6. Team Coordination Quick Reference Table

Member	Focus Area	Primary Code Files	Key Technical Concept
Member 1	Frontend & UI	<code>*.html</code> , <code>js/data.js</code> , <code>css/style.css</code>	Async/Await Fetch, In-Memory Filtering, DOM Templates, Toasts
Member 2	Backend & APIs	<code>app.py</code> (Routes & Controllers)	REST Architecture, JSON Validation, PyMySQL, Error Rollback
Member 3	Database & Schema	<code>schema.sql</code> , <code>seed.py</code> , MySQL	3NF Normalization, Foreign Key Integrity, ENUMs, INNER JOINS
Member 4	Auth & Security	<code>app.py</code> (Decorators, Werkzeug)	scrypt Hashing, Session Cookies, RBAC Decorators, IDOR Protection